# CSE 598 008
# EECS 498 003
# Lab 10

Tianyiming (Henry) Jing

November 6, 2026

# Reminder

- Project 1 (distributed lock service) due November 9
- Problem Set 4 released November 10

# Today

- Refinement exercise: a primary-backup register
- Questions about Project 1? Ask anytime

# A Primary-Backup Register

```
datatype Event =
    | Write(value: int)
    | Read(value: int)
    | NoOp

// The spec: one register,
// initially 0
datatype Variables =
    Variables(value: int)
```

- The register is kept on two hosts, a primary and a backup, so it survives a crash of the primary
- To write: the primary sends `Update` to the backup, the backup applies it and sends `Ack`, then the primary replies
- One write at a time, and no reads while a write is in progress
- Failover: the primary crashes, and the backup takes over as the new primary

# Steps and Events

| Host | Step | Event |
|---|---|---|
| Primary | BeginWrite(x) | NoOp |
| Backup | ApplyUpdate | Write(x) |
| Primary | CompleteWrite | NoOp |
| Primary | ServeRead | Read(value) |
| | | |
| After failover: | | |
| Backup | WriteAsPrimary(x) | Write(x) |
| Backup | ServeRead | Read(value) |
| | | |
| System | Failover | NoOp |

- `Failover` is given: it models a crash and a perfect failure detector
- It is the only step that changes two hosts at once

# Refinement Proof (from Lecture)

```
lemma RefinementInit(v)
    requires Init(v)
    ensures Inv(v)
    ensures Spec.Init(Abstraction(v))

lemma RefinementNext(v, v', evt)
    requires Next(v, v', evt)
    requires Inv(v)
    ensures Inv(v')
    ensures Spec.Next(Abstraction(v),
                      Abstraction(v'), evt)
```

- Abstraction: the spec state that a state of the system stands for
- Each step of the system is a spec step with the same event
- A stutter step is a spec `NoOp` step: the spec state does not change
- `Inv` is proved along the way, as in Chapter 5

# Exercise

- Starter code: `lab10.dfy` on the course GitHub
  - `UtilitiesLibrary.dfy` is in the same folder
- Part 1: fill in the steps of the primary and the backup
  - The signatures are given; each comment says what the step does and its event
- Part 2: prove that the system refines the spec
  - Write the abstraction function and an invariant
  - Prove `RefinementInit` and `RefinementNext`
- Optional, if you finish early: in lecture, the write takes effect when the backup has it
  - Check it in Dafny: move the `Write` event from `ApplyUpdate` to `CompleteWrite` and see what fails

# Extra - Self-Reference Paradoxes

- In a town, the barber shaves exactly the men who do not shave themselves
- Who shaves the barber?

# Extra - Self-Reference Paradoxes

- If he shaves himself, he does not. If he does not, he does
- So no such barber can exist: the description describes no one
- The same pattern, “P if and only if not P”:
  - “This sentence is false.” (the Liar)
- One attempt to solve these paradoxes: forbid self-reference

https://en.wikipedia.org/wiki/Barber_paradox
https://en.wikipedia.org/wiki/Liar_paradox

# Extra - Self-Reference Paradoxes

- Forbidding self-reference does not solve the problem
- A loop works too: “The next sentence is false. The previous sentence is true.”
  - Neither sentence refers to itself
- Yablo’s paradox has no loop at all: sentence n says “every sentence after this one is false”, for n = 0, 1, 2, …
  - No sentence refers to itself, yet no truth values work
  - Whether it is really free of self-reference is still debated
- Dafny code: `self_reference_paradoxes.dfy` in the extra folder on the course GitHub

https://en.wikipedia.org/wiki/Card_paradox
https://en.wikipedia.org/wiki/Yablo's_paradox